

Структуры и функции

Списки хранят много данных, функции наводят порядок в коде. А в конце — игра на двоих.

Четверть 2 · 7 уроков

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 8.** Списки: много данных в одном ящике
2. **Урок 9.** Списки и циклы вместе
3. **Урок 10.** Функции: наводим порядок
4. **Урок 11.** Функции возвращают результат
5. **Урок 12.** Словари: ключ и значение
6. **Урок 13.** 🎮 Проект «Крестики-нолики»: поле
7. **Урок 14.** 🎮 «Крестики-нолики»: игра и защита

Списки: много данных в одном ящике

ЧЕМУ НАУЧИМСЯ

- Хранить много значений в одном списке
- Брать элементы по номеру и добавлять новые командой **append**
- Узнавать длину списка



Зачем нужны списки

Пока в каждый ящик-переменную мы клали **одно** значение. А если нужно хранить пять оценок? Заводить `g1`, `g2`, `g3`, `g4`, `g5` — неудобно. Есть решение лучше: **список** — один ящик, куда помещается сразу много значений.



На пальцах. Переменная — это один ящик. Список — это полка с пронумерованными ячейками. Всё лежит вместе, а каждая ячейка со своим номером. Считаем с нуля: первая ячейка — номер 0.



Разбираем код

Список создаётся квадратными скобками, элементы через запятую:

```
spiski.py

fruits = ["яблоко", "банан", "дыня"]

print(fruits[0])      # яблоко — первый
print(fruits[2])      # дыня — третий
print(len(fruits))    # 3 — сколько всего

fruits.append("виноград") # добавить в конец
print(fruits)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
яблоко
дыня
3
['яблоко', 'банан', 'дыня', 'виноград']
```

Как и у строк, счёт с нуля: `fruits[0]` — первый элемент. `len` говорит, сколько всего элементов. А `append` добавляет новый в конец списка.

ВАЖНО ЗАПОМНИТЬ

Список — в квадратных скобках `[]`. Элементы нумеруются с нуля. `append` — «добавь в конец». `len(список)` — сколько элементов внутри.

ЗАДАНИЕ В ПАРЕ

Список любимого

Создайте список из трёх любимых блюд. Выведите первое и последнее (подсказка: последнее — это `[-1]`). Добавьте в список четвёртое блюдо и выведите весь список.

Водитель печатает, штурман диктует. Через 10 минут — меняетесь.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
fruits[1] # первый
```

```
fruits.append "манго"
```

А ТАК ПРАВИЛЬНО

```
fruits[0] # первый
```

```
fruits.append("манго")
```

Счёт с нуля, как у строк. И у `append` обязательны скобки с тем, что добавляем.

ДОМАШНЕЕ ЗАДАНИЕ

1. Составь список из 5 городов Таджикистана и выведи каждый.
2. Заведи пустой список `[]`, добавь в него три числа через `append`.

★ **Со звёздочкой:** список дел на день — программа спрашивает 3 дела и складывает их в список.

Списки и циклы: обрабатываем данные

ЧЕМУ НАУЧИМСЯ

- Перебирать список циклом **for**
- Считать сумму, среднее, находить максимум
- Обрабатывать оценки целого класса



Список + цикл = сила

На уроке 5 мы перебирали числа через `range`. Но `for` умеет перебирать и **список напрямую** — брать по очереди каждый элемент. Это соединяет две вещи, которые мы уже знаем, и открывает настоящую обработку данных.



На пальцах. Представь стопку тетрадей. Цикл `for` берёт по одной тетради сверху, что-то с ней делает, кладёт, берёт следующую — пока стопка не кончится. Список — это стопка, `for` — твои руки.



Разбираем код

Считаем средний балл по оценкам класса:

```
ocenki.py

grades = [5, 4, 5, 3, 4]

total = 0
for grade in grades:
    total = total + grade

print("Сумма:", total)
print("Средний балл:", total / len(grades))
print("Лучшая оценка:", max(grades))
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
Сумма: 21
Средний балл: 4.2
Лучшая оценка: 5
```

Цикл берёт каждую оценку и прибавляет к `total`. После цикла делим сумму на количество — получаем среднее. А `max` сам находит наибольшее число в списке (есть и `min` — наименьшее).

ВАЖНО ЗАПОМНИТЬ

`for x in список` — берёт каждый элемент по очереди. Чтобы копить сумму, заведи ящик `total = 0` до цикла. Готовые помощники: `max`, `min`, `sum`.

👉 ЗАДАНИЕ В ПАРЕ

Магазинная корзина

Есть список цен: `[12, 45, 8, 30, 15]`. Посчитайте общую стоимость покупки и выведите самый дорогой товар. Подсказка: `sum(...)` и `max(...)`.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
for grade in grades:
    total = grade
```

```
total = 0 # внутри цикла
```

А ТАК ПРАВИЛЬНО

```
for grade in grades:
    total = total + grade
```

```
total = 0 # до цикла!
```

Ящик для суммы заводится **до** цикла, иначе он обнуляется каждый раз. И прибавляй к `total`, а не перезаписывай.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

- В списке температур `[18, 22, 25, 19, 30]` найди среднюю, самую высокую и низкую.
- Посчитай, сколько в списке оценок «пятёрок» (подсказка: `if` внутри цикла + счётчик).

★ **Со звёздочкой:** выведи только те оценки из списка, которые выше среднего балла.

Функции: наводим порядок в коде

ЧЕМУ НАУЧИМСЯ

- Создавать свои команды — функции — словом **def**
- Передавать в функцию данные (аргументы)
- Не повторять один и тот же код по многу раз



Зачем нужны функции

Мы уже пользовались готовыми командами: `print`, `input`, `len`. Это **функции** — маленькие помощники, которые делают одно дело. Оказывается, свои функции можно создавать самому — и это главный способ навести порядок в коде.



На пальцах. Функция — это рецепт с названием. Один раз записал «как заварить чай», а потом просто говоришь «завари чай» — и не пересказываешь все шаги заново. Так и в коде: описал функцию раз — вызываешь её сколько угодно.



Разбираем код

Создаём свою функцию-приветствие словом **def**:

```
funkcii.py

def privet(name):
    print("Привет, " + name + "!")
    print("Рады тебя видеть.")

privet("Алишер")
privet("Фарход")
privet("Мадина")
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
Привет, Алишер!
Рады тебя видеть.
Привет, Фарход!
Рады тебя видеть.
Привет, Мадина!
Рады тебя видеть.
```

Строка `def privet(name):` описывает функцию: её имя `privet` и что она принимает — `name`. Тело функции — с отступом. Дальше мы **вызываем** её три раза с разными именами — и код внутри не пришлось повторять.

ВАЖНО ЗАПОМНИТЬ

`def имя(данные):` — создаём функцию. Тело — с отступом. То, что в скобках при создании — **параметр**, то, что передаём при вызове — **аргумент**. Функцию сначала описывают, потом вызывают.

👉 ЗАДАНИЕ В ПАРЕ

Функция-рамка

Напишите функцию `ramka(text)`, которая печатает текст в рамке из звёздочек. Вызовите её с тремя разными словами.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
def privet(name)
```

```
def privet(name):  
print(name)
```

А ТАК ПРАВИЛЬНО

```
def privet(name):
```

```
def privet(name):  
    print(name)
```

Двоеточие после строки `def`, тело — с отступом. И не забудь вызвать функцию — само по себе описание ничего не печатает.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Функция `privetstvie(name, city)` — здоровается и называет город.
2. Функция `ploshad(a, b)` — печатает площадь прямоугольника.

★ **Со звёздочкой:** функция рисует треугольник из звёздочек заданной высоты.

Функции возвращают результат

ЧЕМУ НАУЧИМСЯ

- Получать из функции результат словом **return**
- Понимать разницу между «напечатать» и «вернуть»
- Собирать программу из функций-помощников



Печатать или вернуть?

Функции с прошлого урока сами печатали результат. Но чаще нужно, чтобы функция **отдала** ответ обратно — и мы решили, что с ним делать: напечатать, сохранить в ящик, использовать в расчёте. За это отвечает слово **return**.



На пальцах. `print` — это когда повар выносит блюдо и сразу ставит на стол. `return` — когда повар отдаёт блюдо тебе в руки, а ты сам решаешь: съесть, отнести другу или положить в холодильник. `return` гибче.



Разбираем код

```
return.py

def summa(a, b):
    return a + b

# результат можно сохранить в ящик
result = summa(10, 5)
print(result)           # 15

# или сразу использовать в расчёте
print(summa(3, 4) * 2)  # 14
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
15
14
```

Функция `summa` не печатает — она **возвращает** число. Поэтому его можно сохранить в `result`, умножить, передать дальше. В этом сила `return`: результат становится обычным значением, с которым можно делать что угодно.

ВАЖНО ЗАПОМНИТЬ

`return` отдаёт результат из функции наружу. После `return` функция заканчивает работу.
Разница: `print` показывает на экране, `return` отдаёт значение, которое можно использовать.

👉 ЗАДАНИЕ В ПАРЕ

Калькулятор на функциях

Напишите функции `summa(a,b)`, `raznost(a,b)`, `proizvedenie(a,b)` — каждая возвращает результат. Затем спросите два числа и покажите все три действия.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
def summa(a, b):  
    print(a + b)  
x = summa(2, 3) # x пустой
```

А ТАК ПРАВИЛЬНО

```
def summa(a, b):  
    return a + b  
x = summa(2, 3) # x = 5
```

Если функция только печатает (`print`), в ящик ничего не попадёт. Чтобы сохранить результат — нужен `return`.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Функция `srednee(a, b, c)` — возвращает среднее трёх чисел.
2. Функция `max_iz_dvuh(a, b)` — возвращает большее из двух.

★ **Со звёздочкой:** функция `chetnoe(n)` возвращает `True` или `False` — чётное ли число.

Словари: ключ и значение

ЧЕМУ НАУЧИМСЯ

- Хранить пары «ключ — значение» в словаре
- Находить значение по ключу
- Построить телефонную книгу



Когда списка мало

В списке элементы ищут по номеру: `[0]`, `[1]`. Но иногда искать хочется по **имени**: не «третий элемент», а «телефон Алишера». Для этого есть **словарь** — он хранит пары: ключ (имя) и значение (телефон).



На пальцах. Словарь — как настоящий словарь или телефонная книга. Ты не листаешь по номерам страниц — ты ищешь слово и сразу видишь перевод. Ключ — это слово, значение — его перевод.



Разбираем код

Словарь создаётся фигурными скобками, пары через двоеточие:

```
slovari.py

phones = {
    "Алишер": "901-234-567",
    "Мадина": "902-345-678",
    "Фарход": "903-456-789"
}

print(phones["Мадина"])    # находим по имени

phones["Сафар"] = "904-567-890" # добавляем нового
print(phones)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
902-345-678
{'Алишер': '901-234-567', 'Мадина': '902-345-678', ...}
```

Вместо номера, как в списке, мы пишем **ключ** — `phones["Мадина"]` — и сразу получаем значение. Чтобы добавить новую пару, просто присваиваем значение новому ключу.

СПИСОК ПРОТИВ СЛОВАРЯ

Список `[]` — доступ по номеру: `spisok[0]`. Словарь `{ }` — доступ по ключу: `slovar["имя"]`. Список — когда важен порядок, словарь — когда важно найти по названию.

👉 ЗАДАНИЕ В ПАРЕ

Словарь оценок

Создайте словарь, где ключ — предмет, значение — твоя оценка: математика, история, информатика. Выведите оценку по информатике и добавьте новый предмет.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
phones = ["Алишер": "901"]
```

```
phones[0] # первый
```

А ТАК ПРАВИЛЬНО

```
phones = {"Алишер": "901"}
```

```
phones["Алишер"]
```

Словарь — в фигурных скобках `{ }`, а не квадратных. И ищем по ключу, а не по номеру.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Словарь «столицы»: страна — столица (5 пар). Выведи столицу Таджикистана.
2. Словарь-меню кафе: блюдо — цена. Посчитай стоимость двух блюд.

★ **Со звёздочкой:** программа спрашивает имя и ищет его телефон в словаре; если нет — говорит «не найдено».

«Крестики-нолики»: строим поле

ЧТО ПОСТРОИМ СЕГОДНЯ

- Игровое поле 3×3 из списка
- Функцию, которая красиво рисует поле
- Ход игрока: ставим крестик в выбранную клетку

Наш большой проект

«Крестики-нолики» на двоих — это проект на два урока. Сегодня строим **поле и ходы**, на следующем уроке добавим проверку победы и защитим проект. Здесь соберётся всё из этой четверти: списки, функции, циклы.

1	2	3
4	X	6
7	0	9

Клетки пронумерованы 1–9. Игрок называет номер — туда ставится его знак.

Поле и его показ

Поле — это список из 9 клеток. Функция рисует его сеткой 3×3:

```
pole.py

board = [" "] * 9    # 9 пустых клеток

def show(board):
    print(board[0], "|", board[1], "|", board[2])
    print("-----")
    print(board[3], "|", board[4], "|", board[5])
    print("-----")
    print(board[6], "|", board[7], "|", board[8])

show(board)
```

`[" "] * 9` создаёт список из девяти пробелов — пустых клеток. Клетка 0 — верхняя левая.

2 Функция `show` рисует поле

Мы вынесли рисование в функцию — теперь можно показать поле в любой момент одной командой `show(board)`.

Теперь ход игрока — ставим знак в выбранную клетку:

```
pos = int(input("Куда ставишь (1-9)? "))
board[pos - 1] = "X"  # игрок назвал 1, а список с 0
show(board)
```

ВАЖНЫЙ МОМЕНТ

Игрок называет клетки 1–9, а список считает с 0. Поэтому `board[pos - 1]`: вычитаем 1, чтобы номер клетки совпал с номером в списке.

👉 ЗАДАНИЕ В ПАРЕ

Соберите поле

Наберите поле и функцию `show`. Сделайте два хода вручную: поставьте X в клетку 5 и O в клетку 1. Проверьте, что поле рисуется правильно.

На следующем уроке превратим это в полную игру.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Допиши: пусть программа спрашивает клетку и ставит X, затем показывает поле.
2. Подумай: как чередовать X и O между двумя игроками? Запиши идею словами.

★ **Со звёздочкой:** сделай так, чтобы нельзя было поставить знак в уже занятую клетку.

«Крестики-нолики»: игра и защита

ЧТО ДОДЕЛАЕМ

- Чередование игроков X и O
- Проверку победы функцией
- Готовую игру — и защиту проекта



Собираем игру целиком

Соединяем поле с прошлого урока, чередование игроков и проверку победы:

```
krestiki.py

board = [" "] * 9

def show(board):
    print(board[0], "|", board[1], "|", board[2])
    print(board[3], "|", board[4], "|", board[5])
    print(board[6], "|", board[7], "|", board[8])

def winner(b):
    lines = [(0,1,2),(3,4,5),(6,7,8),
             (0,3,6),(1,4,7),(2,5,8),
             (0,4,8),(2,4,6)]
    for a, b2, c in lines:
        if b[a] == b[b2] == b[c] != " ":
            return b[a]
    return None

player = "X"
for turn in range(9):
    show(board)
    pos = int(input(player + ", клетка (1-9): "))
    board[pos - 1] = player
    if winner(board):
        show(board)
        print(player + " победил!")
        break
    player = "O" if player == "X" else "X"
```

Восемь способов победить: 3 ряда, 3 столбца, 2 диагонали. Функция проверяет каждую линию — три одинаковых знака подряд.

2 Цикл на 9 ходов

Максимум 9 клеток — значит 9 ходов. После каждого проверяем победу.

3 Смена игрока

Последняя строка меняет X на O и обратно: `"0" if player == "X" else "X"`.

ИТОГОВЫЙ ПРОЕКТ ЧЕТВЕРТИ

Доделайте, украсьте, защитите

Соберите игру и сыграйте пара на пару. Затем улучшите: добавьте сообщение о ничьей, запретите ходить в занятую клетку, добавьте рамку. На защите (2 минуты) расскажите: как устроено поле, как работает проверка победы, что было сложно.

Оба в паре должны уметь объяснить функцию `winner`.

ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Игра работает: двое ходят по очереди, победа определяется верно.
2. Вы добавили хотя бы одно улучшение (ничья, защита клетки, рамка).
3. Каждый может объяснить, как функция `winner` находит победителя.

★ **Со звёздочкой:** добавьте счёт — сколько партий выиграл каждый игрок за вечер.